

Задача А. Экзамен на героя

Автор задачи и разработчик: Даниил Орешников

Для начала заметим, что наличие и a и c в условии — избыточная информация. Задача остается эквивалентной исходной, если изначально $a = 0$, а требуется получить число $c - a$, так как важно только то, сколько в итоге будет добавлено.

Группа 1

В этой группе гарантируется, что изначально $b \leq 4$. Можно просто выписать на бумажке все возможные сценарии, и заметить, что достижимые значения c для всех возможных b — это:

- только 1 при $b = 1$;
- 2 при $(b = 2) \rightarrow (\lfloor \frac{2}{4} \rfloor = 0)$;
- 3 при $(b = 2) \rightarrow (\lfloor \frac{2}{2} \rfloor = 1) \rightarrow (\lfloor \frac{1}{2} \rfloor = 0)$;
- 3 при $(b = 3) \rightarrow (\lfloor \frac{3}{4} \rfloor = 0)$;
- 4 при $(b = 3) \rightarrow (\lfloor \frac{3}{2} \rfloor = 1) \rightarrow (\lfloor \frac{1}{2} \rfloor = 0)$;
- 5 при $(b = 4) \rightarrow (\lfloor \frac{4}{4} \rfloor = 1) \rightarrow (\lfloor \frac{1}{2} \rfloor = 0)$;
- 6 при $(b = 4) \rightarrow (\lfloor \frac{4}{2} \rfloor = 2) \rightarrow (\lfloor \frac{2}{4} \rfloor = 0)$;
- 7 при $(b = 4) \rightarrow (\lfloor \frac{4}{2} \rfloor = 2) \rightarrow (\lfloor \frac{2}{2} \rfloor = 1) \rightarrow (\lfloor \frac{1}{2} \rfloor = 0)$.

Поэтому дальше было достаточно проверить, что изначально $c - a$ равно одному из достижимых значений для данного во вводе b .

Группы 2 и 3

Во второй группе тестов было дано, что $b \leq 10^5$. Можно написать простой перебор всех возможных сценариев, который выглядит как

```
check(a, b, c):  
    if b == 0: return a == c  
    check(a + b, b / 2, c)  
    check(a + b, b / 4, c)
```

Если оценить время работы этого алгоритма, то получается порядка $\mathcal{O}(b)$, так что во второй группе тестов было достаточно написать ровно это.

Если перебор был написан аккуратно и эффективно (например, с проверкой `if a > c: return false`), то его время работы оценивалось как $T(n) = T(\frac{n}{2}) + T(\frac{n}{4}) + 1$. Решение этого соотношения дает сублиненное время работы ($T(b) = o(b)$). И такой перебор уже проходил и третью группу тестов: для $b = 10^8$ решение этого рекуррентного соотношения дает достаточно небольшие значения, чтобы уложиться в ограничения.

Наблюдение

Четвертая и пятая группы тестов позволяли заметить определенное ключевое наблюдение, необходимое для полного решения. Рассмотрим два крайних сценария: всегда делить на 4 и всегда делить на 2. В первом случае мы прибавим

$$\Delta_4 = b + \left\lfloor \frac{b}{4} \right\rfloor + \left\lfloor \frac{b}{16} \right\rfloor + \dots$$

Во втором:

$$\Delta_2 = b + \left\lfloor \frac{b}{2} \right\rfloor + \left\lfloor \frac{b}{4} \right\rfloor + \dots$$

Заметим, что оба значения ограничены сверху такими же суммами, но без округления вниз, в частности $\Delta_4 \leq b + \frac{b}{4} + \frac{b}{16} + \dots = \frac{4}{3}b$ по сумме геометрической прогрессии, и аналогично $\Delta_2 \leq b + \frac{b}{2} + \frac{b}{4} + \dots = 2b$. При этом в точности $2b$ мы на самом деле не можем получить, так как для округления вниз обрезаем как минимум бесконечно много нецелых членов этих двух прогрессий.

Тогда теоретический максимум для случая, когда мы всегда делим на 4, равен $\frac{4}{3}b - 1$. На самом деле теоретический *минимум* не сильно меньше: для любого x верно, что $\lfloor x \rfloor > x - 1$. Всего слагаемых в сумме для Δ_4 ровно $\lfloor \log_4 b \rfloor$, значит $\Delta_4 > \frac{4}{3}b - \log_4 b$.

Итого мы получили

$$\Delta_{\min} = \frac{4}{3}b - \log_4 b, \quad \Delta_{\max} = 2b - 1.$$

Группа 4

В четвертой группе дано, что $c = a + 2b - 1$. Иными словами, необходимо прибавить в точности $2b - 1$, чтобы получить то, что требуется. Заметим, что $2b - 1$ можно получить одним единственным образом (кроме случая $b = 1$): всегда делить только на 2, так как это теоретический достижимый максимум $\Delta_{\max}$. Таким образом, если мы хоть раз поделим на 4 до того, как получим $b \leq 1$, мы уже не сможем этот максимум достичь.

Этого уже достаточно для решения: достаточно проэмулировать процесс. Но можно было решить эту группу вообще за $\mathcal{O}(1)$, если заметить, что теоретический максимум в $2b - 1$ достигается только когда b — степень двойки. Иначе хотя бы одно округление вниз «заберет» еще больше из итоговой суммы.

Группа 5

В пятой группе идея похожая: оценка на $c - a$ близка к теоретическому минимуму $\Delta_{\min}$. Таким образом, в пятой группе *почти* всегда надо делить на 4, и только если этого не хватит, можно запустить небольшой перебор по последним нескольким шагам.

Полное решение

Для полного решения осталось только заметить, что достижимые интервалы чисел при делении текущего b на 4 и делении на 2 вообще не пересекаются. Действительно, мы уже выяснили, что достижимый максимум — это $2b - 1$, а потенциально достижимый минимум — порядка $\frac{4}{3}b - \log_4 b$.

Рассмотрим произвольный момент процесса, и выполним прибавление $a \leftarrow a + b$. Проанализируем, какие значения вообще останутся достижимыми при делении на 2 и на 4.

- Если мы сделаем $b' = \lfloor \frac{b}{4} \rfloor$, *максимальное* значение того, что еще будет прибавлено, равно $2b' - 1 \leq \frac{b}{2}$.
- Если мы сделаем $b' = \lfloor \frac{b}{2} \rfloor$, *минимальное* значение того, что еще будет прибавлено, равно $\frac{4}{3}b' - \log_4 b' \approx \frac{2}{3}b - \log_4 b + \mathcal{O}(1)$. Это всегда больше, чем $\frac{b}{2}$ (для достаточно больших b логарифм не влияет, для маленьких проверяется эмпирически).

Более просто сформулировать это можно так: если мы должны были делить на 2, но поделили на 4, нам уже гарантированно не хватит на нужную сумму; если же мы должны были делить на 4, а поделили на 2, то мы гарантированно взяли больше, чем надо было. Поэтому следующее действие **определяется однозначно**.

Иными словами, корректная логика решения следующая:

1. если $c - a - b \geq b$, то решения гарантированно нет;
2. если $c - a - b > \frac{b}{2}$, то надо делить на 2;
3. если $c - a - b \leq \frac{b}{2}$, то надо делить на 4.

Единственные исключения составляют достаточно небольшие b , для которых не выполняется $\frac{b}{2} < \frac{2}{3}b - \log_4 b$, но как показывает небольшой стресс-тест, таких значений на самом деле вообще не существует. Также этот разбор сделан с уклоном в формальное доказательство, но если не хотелось проверять это неравенство или вообще его выводить, то для решения достаточно было либо оставить небольшой полный перебор действий для небольших b , либо сделать еще более грубые оценки, основанные на более простых наблюдениях.

Итоговое полное решение работает за $\mathcal{O}(\log b)$: каждый следующий шаг определяется однозначно. И, на самом деле, если уже был написан полный перебор (третья группа тестов), его можно было превратить в полное решение добавлением двух отсечений (текущего b либо гарантированно слишком много, либо слишком мало):

```
check(a, b, c):
    if a > c: return false
    if b == 0: return a == c

    if c - a >= 2 * b: return false // недостаточно
    if c - a <= 3/4 * b - log4(b): return false // слишком много

    check(a + b, b / 2, c)
    check(a + b, b / 4, c)
```

P.S. Забавный факт, что даже условие $c - a \leq \frac{4}{3}b - \log_4 b$ на самом деле проверять не обязательно, так как следующих слоях рекурсии большинство попыток делить на два будут отсекаются по $a > c$.

Задача В. Добрый эксперимент

Автор задачи: Антон Вдовин, разработчики: Даниил Орешников и Антон Вдовин

Сначала заметим важный факт, который будет использоваться почти везде: для любой открывающей скобки в ПСП существует отрезок, начинающийся в этой позиции и являющийся ПСП, а для любой закрывающей скобки такого отрезка не существует, потому что никакая ПСП не может начинаться с символа ')'. При внимательном рассмотрении первый факт становится очевидным: для каждой открывающей скобки справа есть «парная» закрывающая, отрезок между ними также является ПСП.

Дальше разберём решения по подгруппам, постепенно улучшая их.

Группа 1

Здесь $n = 6$, а число правильных скобочных последовательностей длины 6 равно 5 (Числа Каталана). Поэтому можно просто перебрать все возможные ПСП длины 6, построив их, например, вручную, и вбив к код как константы.

После этого можно проверить каждого «кандидата»: спросить ответы для всех возможных отрезков. Таких отрезков ровно $\frac{n(n+1)}{2} = 21$. Возьмем каждого кандидата, сравним каждый его подотрезок на соответствие ответу интерактора на этом подотрезке, и если ответы расходятся, то эта строка не является ответом. Итого нам нужно проверить 21 отрезков в 4 из 5 кандидатах (если ни один из четырех не подошел, то ответ пятый), суммарно выходит 84 запроса.

Группа 2

Вторая группа решается так же, как и первая, за исключением того, что нужно придумать как сгенерировать все ПСП длины 10. Это делается стандартным рекурсивным перебором: то, что обозначено в условии как «глубина» ПСП, стандартно называется *балансом*. Альтернативное определение ПСП заключается в том, что суммарный баланс равен 0 (у каждой скобки есть парная) и минимальный баланс тоже равен 0 (ни на каком префиксе не может быть больше открывающих скобок, чем закрывающих). Поэтому в рекурсивный генератор достаточно передавать префикс и текущий баланс, а затем перебирать, какую скобку ставить следующей.

Группа 3

Здесь $n = 22$, $q = 59000$. Перебор всё ещё проходит. А количество ПСП длины 22 равно числу Каталана $C_{11} = 58786$.

Заметим, что число различных ПСП чуть меньше допустимого количества запросов. Давайте научимся за один запрос отбрасывать одного кандидата на ответ. Среди кандидатов выберем две любые строки, затем найдем такой подотрезок $[l; r]$, что в одной строке этот подотрезок будет ПСП, а в другой — нет. Такой всегда существует, причем находить его можно как за $O(n^2)$, так и за $O(n)$. Таким образом, сделав запрос на этом отрезке, мы узнаем, какую из этих строк нужно убрать из кандидатов.

Группа 4

В этой группе гарантируется, что глубина (максимальный баланс) последовательности не превосходит 2. Это значит, что текущий баланс при просмотре строки слева направо может принимать только значения 0, 1 и 2.

Рассмотрим, как восстанавливать строку слева направо.

- Если текущий баланс равен 0, то следующий символ обязан быть '(', потому что ПСП не может начинаться с закрывающей скобки (баланс не может опускаться ниже нуля).
- Если текущий баланс равен 2, то следующий символ обязан быть ')', потому что баланс больше двух запрещен.
- Единственный нетривиальный случай — когда текущий баланс равен 1. Но тогда следующий символ можно определить одним запросом к соседней паре.

Действительно, если к позиции $i - 1$ текущий баланс равен 1, то сделаем запрос $[1, i]$. Если следующий символ — закрывающая скобка, баланс станет равен нулю, и интерактор сообщит YES. Иначе, интерактор сообщит NO. Так мы восстанавливаем всю строку слева направо за линейное число запросов.

Группа 5

Здесь $n \leq 500$, а $q = 2 \cdot 10^5$. Этого уже достаточно, чтобы спросить ответы для всех отрезков, потому что их всего $\frac{n(n+1)}{2} \leq 125250$.

После этого строку можно восстановить напрямую. Действительно, любая ПСП начинается на открывающую скобку и заканчивается закрывающей. Таким образом, если на какой-то запрос $[l, r]$ мы получили YES, то $s_l = '('$ и $s_r = ')'$. Утверждается, что если сделать запросы про каждый возможный отрезок, мы так восстановим всю строку.

Действительно, рассмотрим произвольную позицию i . Если на ней стоит открывающая скобка, у нее есть парная закрывающая справа: нам подойдет отрезок между ними, при запросе в него s_i будет выставлено. Аналогично и симметрично для закрывающей. Это уже даёт полное решение пятой группы за $O(n^2)$ запросов и времени.

Группа 6

Заметим, что первый символ любой непустой ПСП обязан быть '(', поэтому достаточно восстановить позиции от 2 до n . Будем восстанавливать позиции по очереди и предположим, что префикс $s_1 s_2 \dots s_{i-1}$ уже восстановлен. Среди найденных открывающих скобок будем отмечать те, которые уже получили себе пару внутри этого префикса.

Далее здесь подойдет любое решение, заимствующее идею полного решения ниже, но менее эффективно реализующее ее: менее эффективный поиск пары к последней открывающей скобке, другой порядок обхода скобок, любое решение за $\omega(n)$ времени, но $o(q^2)$ запросов. В частности можно было написать что-то похожее на «разделяй и властвуй»:

1. чтобы восстановить ПСП $[l, r]$, сначала сделаем запрос $[l + 1, r - 1]$, поймем, была ли она получена добавлением скобок вокруг другой ПСП;
2. если да, перейдем к $[l + 1, r - 1]$;
3. если нет, будем чередовать запросы $[l, l + i]$ и $[r - i, r]$ по возрастанию i , найдем первый индекс разбиения на две ПСП, перейдем рекурсивно к двум частям.

За счет того, что мы идем одновременно и с левого, и с правого конца, при некоторых оптимизациях такое решение могло пройти эту группу.

Наблюдение

Для следующих групп нужен уже более экономный способ. Одной из важных идей является делать запросы на отрезках длины 2: так мы можем определять стоящие рядом открывающую и закрывающую скобку. Осталось разобраться, что делать в случае, например, строки « $(())$ ». На самом деле ничего сильно отличающегося: воспользуемся стандартной идеей проверки правильности скобочной последовательности со стеком.

Будем восстанавливать строку слева направо и хранить стек позиций ещё не нашедших пару открывающих скобок среди уже восстановленных символов. Например, для префикса « $(())(())$ » будем хранить 1 и 4: у первой и третьей открывающих скобок на этих позициях пока нет пары. Тогда верно то, что ближайшая закрывающая скобка будет парной к последней открывающей в стеке.

Сделаем первый запрос в $[1; 2]$. Если ответ YES, то $s_1 = '('$, а $s_2 = ')'$, иначе гарантированно $s_2 = '('$. Продолжим тем же образом: будем каждый новый запрос делать в $[\text{stack.last}(), i]$, где i — первая неизвестная позиция. Ответ интерактора в точности определит s_i .

Группа 7

Возьмем наблюдение выше и реализуем его. Опять же, как и в шестой группе, эта группа тестов допускала неэффективности в реализации: например, вместо хранения стека можно было за линейное время искать парную открывающую к потенциальной закрывающей на еще неизвестной позиции. Таким образом можно было получить решение за $\mathcal{O}(n^2)$ времени и q запросов.

Группа 8

Здесь $q = n$.

Во всех подгруппах, начиная с этой, достаточно было просто эффективно реализовать описанный выше подход со стеком. Поддерживаем стек, делаем запрос $[\text{stack.last}(), i]$:

- если интерактор сообщил YES, то удаляем верхний элемент из стека, и новая скобка — закрывающая;
- иначе, новая скобка открывающая, и мы добавляем её в стек.

Решение работает за $\mathcal{O}(n)$ и $q = n - 1$, но эта группа допускала возможность сделать один лишний запрос при неэффективной реализации.

Группы 9, 10, 11

Теперь требуется уложиться в q от $n - 1$ до $n - 3$. Для начала достаточно реализовать в точности ту идею, которая была описана выше, это дает решение за $q = n - 1$. Посмотрим теперь, где можно срезать углы.

К предыдущему наблюдению добавим ещё одно: последний символ любой непустой ПСП обязательно равен ')', значит, запрос $[n - 1, n]$ не приносит новой информации, и восстанавливать надо только позиции от 2 до $n - 1$. Тем самым число запросов уменьшается до $n - 2$.

Осталось уложиться в $q = n - 3$. Нужно избавиться ещё от одного запроса. Для этого не будем восстанавливать последнюю *пару* символов запросами.

Будем восстанавливать строку стековым способом только до позиции $n - 2$. После этого останутся две последние позиции, пусть текущий баланс перед ними равен b . За два оставшихся символа итоговый баланс должен стать равен нулю. При этом за один символ баланс меняется только на ± 1 , значит за две позиции он может уменьшиться максимум на 2. Следовательно, b не может быть больше 2.

Кроме того, длина уже восстановленного префикса равна $n - 2$, а это чётное число. Последняя скобка нам известна, она закрывающая, а значит предпоследняя однозначно определяется по числу $b - 1$, которое будет равно ± 1 . Итак, достаточно восстанавливать запросами только позиции от 2 до $n - 2$. Это и даёт ровно $n - 3$ запроса.

Итог

В полном решении запросы задаются только для позиций от 2 до $n - 2$, и на каждую такую позицию делается не более одного запроса. Значит, общее число запросов не превышает $n - 3$. Более общую и эффективную логику, которая приводит к не менее оптимальному результату, можно сформулировать следующим образом:

- не делаем запрос, если текущий баланс равен нулю: следующая обязательно открывающая;
- не делаем запрос, если текущий баланс равен $n - i$: **все** оставшиеся скобки обязательно закрывающие.

Задача С. Потенциал Бибоба

Автор задачи и разработчик: Ильнур Валеев

Рассмотрим строку s и разобьём её на максимальные блоки одинаковых подряд идущих символов. Пусть длины блоков равны

$$l_1, l_2, \dots, l_m.$$

Тогда потенциал строки равен

$$l_1^2 + l_2^2 + \dots + l_m^2.$$

Разрешено выполнить не более одной операции: взять один символ, удалить его из строки и вставить в любое место. Важно заметить, что после такой операции изменяются только блок, из которого символ удалили, и блок, в который символ вставили (а также, возможно, их соседи).

Наблюдения

1. Большие блоки — «дорогие».

Если уменьшить блок длины L на 1, вклад в потенциал изменится на

$$(L - 1)^2 - L^2 = -2L + 1.$$

Чем больше L , тем выгоднее забирать символ именно из этого блока.

2. Если вставлять символ внутрь «чужого» блока (другой буквы), блок выгодно делить как можно равномернее.

Пусть блок длины T разбивается на два блока длины x и y , где $x + y = T$. Тогда новый вклад:

$$x^2 + y^2 + 1.$$

При фиксированной сумме $x + y = T$ сумма квадратов минимальна, когда части максимально близки:

$$x = \left\lfloor \frac{T}{2} \right\rfloor, \quad y = \left\lceil \frac{T}{2} \right\rceil.$$

3. Если вставлять символ в блок того же символа, лучше увеличивать самый короткий блок.

Если увеличить блок длины T до $T + 1$, потенциал изменится на

$$(T + 1)^2 - T^2 = 2T + 1,$$

что минимально при минимальном T .

4. Если удалить блок длины 1, он может склеить соседние блоки.

Например:

aa b aaa $\rightarrow$ aaaaaa.

Это нужно учитывать отдельно.

Группа 1

В этой группе тестов суммарная длина строк не превосходит 200.

Можно перебрать все варианты одной операции:

1. выбрать позицию i ;
2. удалить символ;
3. вставить его во все возможные позиции;
4. пересчитать потенциал.

Сложность такого решения — $\mathcal{O}(n^3)$ из перебора удаляемой позиции, перебора места вставки, и, собственно, модификации строки и пересчета потенциала на каждый такой способ. Этого достаточно для этой подзадачи, чтобы уложиться в ограничения по времени.

Группа 2

Суммарная длина строк не превосходит 2000. Оптимизируем предыдущее решение до $\mathcal{O}(n^2)$ по времени.

Заметим, что после одной операции меняются только несколько блоков. Поэтому можно:

1. сначала сжать строку в блоки;
2. перебирать блок-источник и блок вставки;
3. пересчитывать изменение потенциала только локально.

«Пересчитывать локально» в данном случае означает вычесть потенциалы исчезнувших или измененных блоков и добавить потенциалы новых. Это можно делать за $\mathcal{O}(1)$, что уменьшает общее время работы до $\mathcal{O}(n^2)$.

Группа 3

В этой группе тестов в строке не более двух различных букв. В этом случае оптимальная операция может быть только одного из следующих типов:

1. Перенести символ из самого длинного блока в блок другой буквы и разбить его на две максимально равные части.
2. Перенести символ из самого длинного блока в самый короткий блок той же буквы.
3. Удалить блок длины 1, возможно склеив соседние блоки, и затем вставить символ внутрь блока другой буквы.

Все необходимые значения (максимальный блок, минимальный блок, лучший одиночный блок) можно найти за один проход по блокам, после чего ответ считается по формулам за $\mathcal{O}(1)$ с итоговой сложностью $\mathcal{O}(n)$.

Группа 4

Здесь каждый символ образует не более одного блока. Тогда число блоков не превышает размера алфавита (26), поэтому можно просто перебрать:

1. из какого блока забираем символ;
2. в какой блок вставляем;
3. разбиваем ли блок.

Аналогично, каждое изменение приводит к достаточно локальным пересчетам, и время работы равно $\mathcal{O}(n + m^2)$ для $m \leq 26$ — числа блоков.

Полное решение

Пусть текущий потенциал равен

$$S = l_1^2 + l_2^2 + \dots + l_m^2.$$

Рассмотрим все возможные полезные операции.

1. Перенос в блок другой буквы

Берём символ из блока длины $L > 1$ и вставляем в блок длины T другой буквы:

$$S - L^2 + (L - 1)^2 - T^2 + \left\lfloor \frac{T}{2} \right\rfloor^2 + \left\lceil \frac{T}{2} \right\rceil^2 + 1.$$

2. Перенос в блок той же буквы

$$S - L^2 + (L - 1)^2 - T^2 + (T + 1)^2.$$

3. Перенос из одиночного блока

Если блок длины 1 находится между блоками длины a и b одной буквы, то после удаления они склеиваются:

$$(a + b)^2 - a^2 - b^2.$$

После этого вставляем символ внутрь блока длины T :

$$-T^2 + \left\lfloor \frac{T}{2} \right\rfloor^2 + \left\lceil \frac{T}{2} \right\rceil^2.$$

4. Перестроение вокруг одиночного блока

Для фрагмента

$$\mathbf{x}^a \mathbf{y} \mathbf{x}^b$$

можно получить

$$\mathbf{x}^{\lfloor (a+b)/2 \rfloor} \mathbf{y} \mathbf{x}^{\lceil (a+b)/2 \rceil}.$$

Старый вклад:

$$a^2 + 1 + b^2,$$

новый вклад:

$$\left\lfloor \frac{a+b}{2} \right\rfloor^2 + 1 + \left\lceil \frac{a+b}{2} \right\rceil^2.$$

Все необходимые значения можно посчитать за один проход по блокам, а затем перебрать все пары букв. Пользуясь наблюдениями в самом начале достаточно не перебирать все итоговые блоки, а только рассмотреть наиболее оптимальные блоки для вставки: максимальный или минимальный по каждой букве. Итоговая сложность: $\mathcal{O}(n)$ на одну строку.

Задача D. Нетрадиционная игра

Автор задачи и разработчик: Павел Скобелкин

Группы 1 и 2

Для решения первой подзадачи можно было воспользоваться полным рекурсивным перебором случаев, в котором оба игрока будут выбирать для себя лучшие исходы соответственно. Оптимальная стратегия означает, что для каждого игрока надо было перебрать ходы противника, вернуть наиболее оптимальные достижимые противником состояния, и выбрать из всех своих ходов тот, который приводит к своему оптимуму среди рекурсивных результатов.

Для решения второй подзадачи достаточно было добавить меморизацию состояний перебора, заметив, что они часто будут повторяться.

Группы 7, 8, 9

В этих группах тестов было ограничение $k = 1$. Поймем, что будет происходить в таком случае.

Игра будет состоять из одного хода первого игрока, причем он хочет максимизировать сумму массива. В этой и последующих подзадачах обозначим $d_i = (a_i \oplus M) - a_i$ (где $M = 2^x - 1$) — выгода, которую игрок получит, если «возьмет» этот элемент. Финальный счет игры будет равен изначальной сумме массива a плюс сумме на отрезке в массиве выгод. Сумма массива a — константа, получается, в этой подзадаче необходимо было найти подотрезок массива d с максимальной суммой.

В зависимости от эффективности, можно было получить разный балл: седьмую подзадачу проходило решение с асимптотикой $\mathcal{O}(n^3)$: переберем отрезок $[l, r]$ за $\mathcal{O}(n^2)$, посчитаем на нем сумму за $\mathcal{O}(n)$ и возьмем максимум по всем отрезкам.

Решение седьмой подзадачи можно оптимизировать до $\mathcal{O}(n^2)$, используя префиксные суммы, и тем самым пройти **восемью** подзадачу.

Девятую подзадачу можно было пройти, также используя префиксные суммы: $d_l + d_{l+1} + \dots + d_r = \text{pref}_{r+1} - \text{pref}_l$. Таким образом, для максимизации суммы на отрезке, если правая граница фиксирована, нужно выбирать левую границу с минимальной префиксной суммой. Это возможно сделать за $\mathcal{O}(n)$, храня минимумы на префиксных суммах или воспользовавшись любым другим стандартным подходом поиска отрезка максимальной суммы.

Группы 3, 4

Для решения третьей подзадачи можно было воспользоваться полным перебором ходов.

Случай $k = 1$ уже разобран выше; при $k = 2$ для каждого хода первого игрока $[l_1, r_1]$ (таких ходов $\mathcal{O}(n^2)$) найдем оптимальный ход второго игрока. Возможных ходов второго игрока также $\mathcal{O}(n^2)$, после этого счет игры можно посчитать за $\mathcal{O}(n)$. Таким образом, это решение работает за $\mathcal{O}(n^5)$.

Используя технику префиксных сумм, можно оптимизировать предыдущее решение (чтобы не считать счет игры за $\mathcal{O}(n)$). Таким образом получить асимптотику $\mathcal{O}(n^4)$ и пройти четвертую подзадачу.

Группа 5

Далее рассмотрим решение пятой подзадачи за $\mathcal{O}(n^3)$. Также переберем ход первого игрока, и для каждого хода первого игрока найдем самый оптимальный ход второго, но будем искать его более эффективно.

Пусть первый игрок сделал ход $[l, r]$. Тогда есть несколько вариантов того, куда второй игрок будет делать ход:

1. в $[0, l)$, то есть строго слева от первого отрезка;
2. в $(r, n - 1]$, то есть строго справа;
3. в $[l, r]$, то есть внутри первого хода;
4. пересекая ровно одну из двух границ;
5. в отрезок, полностью содержащий $[l, r]$.

Теперь, для пяти возможных вариантов, нужно решить задачу за $\mathcal{O}(n)$. Несложно понять, что первые три варианта аналогичны случаю $k = 1$ на подотрезках массива: решается за линейное время тем же способом. Для вариантов 4 и 5 нужно посчитать оптимальные ходы вида $[i, l]$, $[l, j]$, $[i, r]$ и $[r, j]$, что можно сделать за $\mathcal{O}(n)$ суммарно, просто перебирая i и j по порядку.

Группа 6

Для решения шестой подзадачи можно было заметить, что все вышеописанные значения можно было пересчитывать быстрее.

Давайте зафиксируем левую границу первого хода и будем перебирать правую границу по возрастанию. Научимся пересчитывать минимум во всех пяти случаях.

Случай 3 обрабатывается за $\mathcal{O}(n)$ для фиксированного l и всех r аналогично случаю нечётного k (ищем максимальный по сумме подотрезок на $[l, r]$, пересчитывается за $\mathcal{O}(1)$ с каждым увеличением r на единицу).

Для случаев 1 и 2 нужно научиться считать отрезок с минимальной суммой на префиксе и суффиксе массива d . Это предподсчитывается за $\mathcal{O}(n)$ одним проходом с начала и одним проходом с конца.

В случае 5 начало отрезка l' лежит до l , а конец r' — после r . Таким образом, его сумма имеет вид $\min(d'_{r'} - d'_{l'})$, где $l' \leq l$ и $r' \geq r$. Чтобы находить максимум, достаточно знать минимум и максимум на префиксе и суффиксе, что тоже предподсчитывается. При этом $d'_{l'} = d_{l'}$, а $d'_{r'} = d_{r'} + \Delta$, где Δ — изменение суммы массива b после хода первого игрока.

Случай 4, когда пересекается только одна из двух границ, обрабатывается похоже — например, слева берется все еще максимум, а справа — минимум d' на $[l, r]$, что можно считать в процессе при движении границы r .

Итого решение за $\mathcal{O}(n^2)$.

Группа 10

Для десятой подзадачи заметим, что все элементы массива равны, а значит все значения d_i также равны одному и тому же числу $c = 2^x - 1 - 2a_1$. Так как $2^x - 1$ нечетно, число c не может быть равно нулю.

Если $c > 0$, то переворачивать любой элемент выгодно. Тогда при нечетном k первый игрок просто выбирает весь массив и получает прибавку nc . При четном k второй игрок может повторить ход первого и вернуть массив в исходное состояние, поэтому итоговая прибавка будет равна 0.

Если $c < 0$, то каждый элемент, перевернутый нечетное число раз, только ухудшает ответ для первого игрока. При нечетном k ему выгодно выбрать отрезок длины 1, поэтому ответ равен исходной сумме плюс c .

Разберем случай четного k . Пусть первый игрок выбрал отрезок $[l, r]$. Тогда второй игрок хочет максимизировать количество элементов, перевернутых нечетное число раз.

Он может выбрать, например:

- отрезок $[1, l - 1]$, тогда нечетно перевернутыми останутся r элементов;
- отрезок $[r + 1, n]$, тогда нечетно перевернутыми останутся $n - l + 1$ элементов;
- отрезок $[1, n]$, тогда нечетно перевернутыми останутся все элементы вне $[l, r]$, то есть $n - (r - l + 1)$ элементов.

Следовательно, после выбора первого игрока $[l, r]$ второй может гарантировать не менее чем

$$\max(r, n - l + 1, n - (r - l + 1))$$

нечетно перевернутых элементов. С другой стороны, несложный разбор всех взаимных положений двух отрезков показывает, что больше добиться нельзя.

Значит итог после выбора первого игрока $[l, r]$ равен

$$\text{base} + c \cdot \max(r, n - l + 1, n - (r - l + 1)).$$

Остается минимизировать это выражение по всем l и r . Обозначим

$$a = l - 1, \quad b = n - r, \quad p = r - l + 1.$$

Тогда $a + b + p = n$, а

$$\max(r, n - l + 1, n - (r - l + 1)) = \max(a + p, p + b, a + b) = n - \min(a, b, p).$$

Следовательно, первому игроку нужно максимизировать $\min(a, b, p)$, то есть разбить массив на три части как можно более равномерно. Поэтому

$$\min_{1 \leq l \leq r \leq n} \max(r, n - l + 1, n - (r - l + 1)) = n - \left\lfloor \frac{n}{3} \right\rfloor = \left\lceil \frac{2n}{3} \right\rceil.$$

Итак, для десятой подзадачи ответ равен

$$\text{base} + \begin{cases} nc, & \text{если } k \text{ нечетно и } c > 0, \\ 0, & \text{если } k \text{ четно и } c > 0, \\ c, & \text{если } k \text{ нечетно и } c < 0, \\ c \cdot \left\lceil \frac{2n}{3} \right\rceil, & \text{если } k \text{ четно и } c < 0. \end{cases}$$

Группа 12

В этой подзадаче массив d будет иметь вид $d_1 = c, \quad d_2 = -c, \quad d_3 = c, \quad d_4 = -c, \dots$. Значит сумма на любом отрезке может быть только одной из трех: $c, 0, -c$.

Рассмотрим случай четного k . Так как сумма любого отрезка может быть только равна $c, 0$ или $-c$, а итог после двух ходов равен сумме на симметрической разности двух отрезков, то возможных значений тоже константное число. После перебора всех случаев по четности границ получается следующая формула.

Если $n \leq 2$, то второй игрок всегда может полностью нейтрализовать ход первого, поэтому ответ равен

$$\text{base}.$$

Если $n \geq 3$ нечетно, то ответ равен

$$\text{base} + \begin{cases} 0, & \text{если } c > 0, \\ 2c, & \text{если } c < 0. \end{cases}$$

Если $n \geq 4$ четно, то ответ равен

$$\text{base} - |c|.$$

Группа 14

В этой подзадаче для каждого i выполнено $a_i < 2^{x-1}$. Тогда

$$d_i = (2^x - 1) - 2a_i \geq 1,$$

то есть все значения d_i строго положительны. Поэтому в случае нечетного k первому игроку выгодно перевернуть весь массив, и ответ будет равен

$$\sum_{i=1}^n ((2^x - 1) - a_i) = n(2^x - 1) - \sum_{i=1}^n a_i.$$

При четном k второй игрок может повторить ход первого и вернуть массив в исходное состояние. Поскольку все d_i положительны, никакой ход второго игрока не может сделать итог меньше исходной суммы, значит ответом будет просто исходная сумма массива.

Наблюдение

Здесь же сделаем **важное наблюдение**: случай нечетного k эквивалентен $k = 1$. Аналогично, случай произвольного четного k эквивалентен $k = 2$ по тем же самым причинам. Докажем это свойство формально:

- Пусть k нечетное. Пусть результат игры за один ход это A . Тогда первый может гарантировать себе результат A : делает первый ход, максимизируя сумму, а далее повторяет (отменяет) все ходы соперника. Покажем, что второй тоже может гарантировать себе результат A : он будет просто повторять все ходы первого, тем самым оставив первому один ход в самом конце.
- Пусть k четное. Пусть первый и второй сделали ход — тогда далее первый может повторять ходы второго, тем самым не изменив результат после двух ходов. Аналогично, второй может повторять все ходы первого, кроме последнего хода, тем самым гарантировав себе результат за два хода.

Таким образом, решения групп 3 – 6 также являются решениями групп 15 – 17.

Группа 13

В тринадцатой подзадаче подразумевалось сведение массива большой длины к массиву меньшей длины. Сожмем блоки одинаковых чисел, после будем рассматривать каждое действие как затрагивающее два блока частично и все блоки посередине целиком. Для блоков, затронутых целиком, можно просто хранить и использовать общее d_i . Для крайних блоков число затронутых элементов аккуратно перебирается или оптимально определяется с учетом логики, описанной выше.

Полное решение

Заметим, что случай $k = 1$ уже разобран выше и тривиален: все сводится к поиску отрезка максимальной суммы. Далее будем рассматривать только случай $k = 2$ и эффективно искать ход первого игрока, при котором оптимальный ход второго игрока наиболее выгоден первому.

Пусть первый игрок выбрал отрезок $[l, r]$, а второй — отрезок $[l', r']$. Тогда итоговая прибавка к сумме массива равна сумме d_i по симметрической разности этих двух отрезков. Обозначим через $f(l, r)$ минимальную прибавку, которую сможет оставить второй игрок после хода первого игрока $[l, r]$. Тогда требуется найти $\max_{1 \leq l \leq r \leq n} f(l, r)$.

Будем делать бинарный поиск по ответу. То есть проверять предикат: можно ли первому игроку гарантировать прибавку не меньше, чем X (монотонный предикат).

Итак, остается научиться быстро проверять, существует ли отрезок $[l, r]$, для которого

$$f(l, r) \geq X.$$

Сделаем это с помощью «разделяй и властвуй». Пусть функция $\text{check}(L, R, X)$ проверяет, существует ли подходящий отрезок $[l, r]$, полностью лежащий в $[L, R]$. Если $L = R$, то отрезок только один, и его значение можно посчитать напрямую за $\mathcal{O}(1)$, используя заранее посчитанные минимумы на префиксах и суффиксах.

Иначе положим

$$M = \left\lfloor \frac{L + R}{2} \right\rfloor.$$

Тогда искомый отрезок либо:

1. целиком лежит в $[L, M]$;
2. целиком лежит в $[M + 1, R]$;
3. пересекает середину, то есть имеет вид $[l, r]$, где $l \leq M < r$.

Первые два случая обрабатываются рекурсивно. Остается рассмотреть только отрезки, пересекающие середину.

Для каждой левой части $[l, M]$ построим тройку чисел (A_l, B_l, C_l) :

- A_l — вклад левой части, если второй игрок выбирает отрезок целиком справа от середины;
- B_l — минимальный вклад левой части, если отрезок второго игрока пересекает середину;
- C_l — минимальный вклад левой части, если отрезок второго игрока целиком лежит слева от середины.

Аналогично, для каждой правой части $[M + 1, r]$ построим тройку (A_r, B_r, C_r) :

- A_r — минимальный вклад правой части, если второй игрок целиком справа от середины;
- B_r — минимальный вклад правой части, если второй игрок пересекает середину;
- C_r — вклад правой части, если второй игрок целиком слева от середины.

Тогда для отрезка первого игрока $[l, r]$ второй игрок имеет ровно три типа оптимальных ответов относительно середины:

1. его отрезок целиком слева от середины;
2. его отрезок пересекает середину;
3. его отрезок целиком справа от середины.

Следовательно, первый игрок может гарантировать значение не меньше X тогда и только тогда, когда одновременно выполнены три неравенства

$$A_l + A_r \geq X, \quad B_l + B_r \geq X, \quad C_l + C_r \geq X.$$

Значения (A_l, B_l, C_l) для всех $l = M, M - 1, \dots, L$ и значения (A_r, B_r, C_r) для всех $r = M + 1, M + 2, \dots, R$ можно посчитать за линейное время, если двигать границы по одному и поддерживать:

- сумму текущего отрезка;
- максимум суммы подотрезка внутри него;
- минимальный префикс;
- минимальный суффикс;
- а также заранее посчитанные минимумы на префиксах и суффиксах всего массива.

Теперь задача для пересекающих середину отрезков свелась к следующей: даны все левые тройки (A_l, B_l, C_l) и все правые тройки (A_r, B_r, C_r) ; нужно проверить, существует ли пара, для которой

$$A_l \geq X - A_r, \quad B_l \geq X - B_r, \quad C_l \geq X - C_r.$$

Это стандартная задача о доминировании в трех измерениях.

Решим ее так:

1. отсортируем левые тройки по первой координате A по убыванию;
2. для каждой правой тройки создадим запрос с порогами

$$(X - A_r, X - B_r, X - C_r);$$

3. отсортируем запросы по первой координате по убыванию;

4. будем sweep-line'ом добавлять в структуру все левые тройки, у которых первая координата уже достаточна;
5. по второй координате сделаем сжатие координат, а в Fenwick tree будем хранить максимум третьей координаты среди уже добавленных троек.

Тогда для каждого запроса можно за $\mathcal{O}(\log n)$ проверить, существует ли добавленная левая тройка, у которой одновременно

$$B_l \geq X - B_r \quad \text{и} \quad C_l \geq X - C_r.$$

Значит все пересекающие середину отрезки проверяются за

$$\mathcal{O}((R - L + 1) \log(R - L + 1)).$$

Итак, одна проверка `check(L, R, X)` работает за

$$\mathcal{O}(n \log^2 n) :$$

на каждом уровне рекурсии суммарный размер всех отрезков равен $\mathcal{O}(n)$, а работа на одном узле стоит $\mathcal{O}(\text{len} \log \text{len})$.

Остается сделать бинарный поиск по ответу. Прибавка для случая двух ходов не может быть больше 0, потому что второй игрок всегда может повторить ход первого. С другой стороны, она не меньше, чем

$$-\sum_{i=1}^n |d_i|.$$

Значит бинарный поиск идет по отрезку

$$\left[-\sum |d_i|, 0 \right].$$

Итоговая асимптотика: $\mathcal{O}(n \log^2 n \log C)$, где $C = \sum |d_i|$.

Задача Е. Врата Вавилона

Автор задачи и разработчик: Егор Юлин

Группа 1

Для решения данной подзадачи можно было перебрать стартовое значение силы, оно равно H или $H - h_1$, после чего проэмулировать сражение со Слугой, и повторно перебрать применение артефакта.

Группа 2

Для решения данной подзадачи можно было при фиксированном d перебрать все достижимые значения силы и проэмулировать все сражения со Слугами, полностью перебирая все возможные варианты действий на каждой фазе.

Группы 3, 4, 5, 8

Для решения данных подзадач можно было использовать динамическое программирование. Будем хранить каждую достижимую силу, обозначим достижимость за $\text{dp}[x]$, где данное значение равно 1, если король героев перед битвой может иметь x силы, и 0 в противном случае.

Тогда пересчет можно делать с помощью обхода в глубину, перебирая для одного x различные артефакты: проведем ребро из силы i в силу j , если применением одного артефакта, имея силу i , мы получаем силу ровно j . Для каждого боя будем делать переход «если можно было иметь x силы,

то теперь достижима $x - a_i$ ». Для каждой фазы восстановления будем заново запускать обход в глубину всех доступных состояний и обновлять `dp`.

Итоговое время работы будет $\mathcal{O}(nmH)$, что подходит под данные ограничения. В подгруппах с меньшими ограничениями можно было слегка упростить или менее эффективно реализовать ту же самую логику.

Группы 6, 7

Для решения данных подзадач можно было использовать `std::bitset` для хранения и быстрого пересчета достижимых значений силы за время $\mathcal{O}\left(\frac{nmH}{64}\right)$, что укладывается в ограничения по времени в данных подзадачах. Операция сражения — это просто битовый сдвиг битсета вправо, а применение артефакта — сдвиг влево, плюс после дополнительно учесть действие тех артефактов, остаточная сила которых уходит в минус.

Наблюдение

Заметим, что если хранить достижимую силу не просто как силу, а как недостоящую до максимума силу, то есть хранить $x' = H - x$, то тогда операция применения артефакта эквивалентна следующему переходу: $x' \rightarrow |x' - h_i|$. Это работает и для случая неперевышения H , и для случая, когда артефакт работает сначала в плюс, потом в минус.

Тогда так же заметим, что все переходы будут эквивалентны взятию остатка по модулю $\text{gcd}(h_1, \dots, h_n)$ по теореме Безу, так как все $h_i \leq H$ и после применения артефакта сила не опускается ниже 0 (при $x = H$ и $h_i = H$ нет смысла применять артефакт).

Тогда мы можем считать, что у нас есть ровно один артефакт, сила которого равна $D = \text{gcd}(h_1, \dots, h_n)$. Более того, достижимую силу можно считать по этому модулю, потому что с таким артефактом можно получить любое число от 0 до H с тем же остатком по модулю D .

Группа 9

Если применить выше описанные размышления и алгоритм для подгрупп 3 – 5, мы можем получить время работы $\mathcal{O}(n \log H + mH)$.

Полное решение

Будем использовать наблюдение выше и алгоритм для подгрупп 3 – 5, но теперь достижимость снова будем хранить в `std::bitset`. Тогда при сражении со Слугой необходимо научиться обрабатывать несколько случаев:

- Если $\text{gcd}(h_1, \dots, h_m) + a_i \leq H$, то вся достижимая сила просто циклически сдвинется после сражения со Слугой. Теперь необходимо обновить достижимую силу с учетом фазы отдыха. Так как мы храним достижимую силу в диапазоне $[0; \text{gcd}(h_1, \dots, h_n))$, то для любой силы x и артефакта i верно $x \leq h_i$.

Тогда мы так же можем заменить $x \rightarrow -x \bmod \text{gcd}(h_1, \dots, h_n)$, то есть наложить перевернутый битсет к состояниям достижимости. Данная операция даст нам все достижимые состояния силы после сражения со Слугой, и тогда необходимо будет вывести максимальное значение $H - x$, то есть минимальное x , хранимое в битсете;

- Если $\text{gcd}(h_1, \dots, h_m) + a_i > H$, то несколько крайних значений силы нельзя использовать для битвы, так как они будут не больше, чем a_i , тогда можно занулить граничные значения в битсете, сделав сдвиг битсета на соответствующую величину, после чего провести сражения, как будто $\text{gcd}(h_1, \dots, h_m) + a_i \leq H$.

Тогда время работы данного алгоритма будет равно $\mathcal{O}\left(n \log H + \frac{mH}{64}\right)$, что укладывается в ограничения по времени.

Задача F. Непалиндромное заклинание

Авторы задачи и разработчики: Ильдар Гайнуллин и Андрей Матвеев

Далее под «путём» мы будем понимать **маршрут**: вершины и рёбра разрешается посещать несколько раз.

Основное наблюдение

Обозначим через L максимальную длину маршрута, в символах на ребрах которого встречаются **хотя бы две различные буквы**.

Тогда искомым ответ ans всегда удовлетворяет

$$\text{ans} \in \{L, L - 1\}.$$

Действительно, любой непалиндром содержит хотя бы две разные буквы, значит

$$\text{ans} \leq L.$$

Возьмём маршрут длины L . Если его слово уже не является палиндромом, то $\text{ans} = L$. Иначе это палиндром, но в нём есть хотя бы две разные буквы. Тогда хотя бы один из двух отрезков длины $L - 1$ — префикс или суффикс — не является палиндромом. Иначе оба они были бы палиндромами, откуда легко следует, что все символы исходного слова одинаковы, что невозможно.

Итак:

$$\text{ans} = \begin{cases} -1, & \text{если } L = \infty, \\ 0, & \text{если не существует маршрута, содержащего хотя бы две разные буквы,} \\ L \text{ или } L - 1, & \text{иначе.} \end{cases}$$

Таким образом, задача распадается на две части:

- найти L или понять, что $L = \infty$;
- в конечном случае определить, равен ответ L или $L - 1$.

Группа 1

При $n, m \leq 5$ проходит аккуратный полный перебор, например достаточно перебрать все пути длины $\leq n + 2$, и если есть путь длины $> n$, то ответ бесконечный. А иначе, путей достаточно мало и можно просто взять из них самый длинный, проверяя на палиндромность наивным образом.

Группы 2, 3, 4

Здесь удобно сделать динамику по парам вершин.

Пусть $\text{mid}[u][v]$ — максимальная длина **произвольного** маршрута из u в v . Значение может быть:

- не существует;
- конечное число;
- бесконечность.

Сначала сожмём граф в компоненты сильной связности. Если на каком-то пути из u в v встречается компонента с циклом, то $\text{mid}[u][v] = \infty$. Иначе все маршруты лежат в DAG'е, и максимум считается обычной динамикой по DAG'у.

Теперь определим $\text{base}[u][v]$ — максимальную длину непалиндромного маршрута из u в v , у которого **первая** и **последняя** буквы уже различны. Тогда

$$\text{base}[u][v] = \max(2 + \text{mid}[x][y]),$$

где максимум берётся по всем рёбрам

$$u \xrightarrow{a} x, \quad y \xrightarrow{b} v, \quad a \neq b.$$

Если какой-то из таких $\text{mid}[x][y] = \infty$, то и $\text{base}[u][v] = \infty$.

Теперь построим граф состояний (u, v) . Переход

$$(u, v) \rightarrow (x, y)$$

существует, если найдётся буква c такая, что есть рёбра

$$u \xrightarrow{c} x, \quad y \xrightarrow{c} v.$$

Смысл такой: если слово начинается и заканчивается одной и той же буквой c , то эти две буквы можно удалить, и мы попадём в состояние (x, y) .

Тогда для $\text{dp}[u][v]$ — максимальной длины непалиндромного маршрута из u в v — верно:

$$\text{dp}[u][v] = \max\left(\text{base}[u][v], 2 + \max \text{dp}[x][y]\right),$$

где максимум берётся по всем переходам $(u, v) \rightarrow (x, y)$.

Бесконечность возникает в двух случаях:

- существует состояние с $\text{base}[u][v] = \infty$;
- в графе состояний (u, v) есть цикл, из которого можно добраться до состояния, где существует подходящая база.

Иначе этот граф можно свести к DAG'у и посчитать dp в порядке топологической сортировки. Этого достаточно для ограничений $n, m \leq 200$. Время работы $\mathcal{O}(n(n + m) + m^2)$.

Группа 5

В этой подзадаче известно, что ответ либо нечётный, либо равен 0, либо равен -1 . Значит достаточно найти только L .

Построим автомат по множеству уже встреченных букв. Для каждой вершины v исходного графа создадим состояния:

- (v, S) — пока не прошли ни одного ребра;
- (v, C_x) — пока встречалась только буква x ;
- (v, M) — уже встретились хотя бы две разные буквы.

Для каждого ребра $u \xrightarrow{c} v$ добавим переходы автомата:

- $(u, S) \rightarrow (v, C_c)$;
- $(u, C_c) \rightarrow (v, C_c)$;
- $(u, C_x) \rightarrow (v, M)$ для всех $x \neq c$;
- $(u, M) \rightarrow (v, M)$.

Стартовыми считаем все состояния (v, S) , а финишными — все состояния (v, M) .

Теперь длина маршрута в автомате равна длине маршрута в исходном графе, а маршрут заканчивается в состоянии M тогда и только тогда, когда в его слове встретились хотя бы две разные буквы.

Оставим только те вершины автомата, которые достижимы из старта и из которых достигим финиш.

- Если в этом подграфе есть цикл, то $L = \infty$, а значит ответ равен -1 .
- Иначе это DAG, и L ищется как максимальный путь в DAG'е.

После этого ответ восстанавливается сразу:

- если маршрута с двумя разными буквами не существует, то ответ 0;
- если L нечётно, то ответ L ;
- если L чётно, то ответ $L - 1$.

Группа 6

Здесь предполагается более аккуратно написанное решение прошлых подзадач.

Группа 7

Здесь для всех рёбер выполняется $u_i < v_i$, то есть исходный граф уже является DAG'ом.

Отсюда сразу следует, что бесконечного ответа быть не может. Автомат из предыдущего пункта тоже будет DAG'ом, так как каждое ребро по-прежнему идёт в вершину с большим номером.

Дальше:

- находим L в автомате;
- оставляем только рёбра, лежащие на каком-то максимальном пути длины L ;
- проверяем, существует ли среди этих путей непалиндром.

Проверка делается через середину маршрута; тот же приём будет использован и в полном решении ниже.

Группа 8

В этой подзадаче алфавит бинарный, то есть встречаются только буквы 'a' и 'b'.

Поэтому автомат сильно упрощается: для каждой вершины достаточно всего 4 состояния:

- S — ещё не прошли ни одного ребра;
- A — пока встречали только 'a';
- B — пока встречали только 'b';
- AB — уже встретили и 'a', и 'b'.

Дальше всё полностью совпадает с полным решением: находим L , а затем решаем, равен ли ответ L или $L - 1$.

Полное решение

Распишем по шагам все необходимые идеи:

1. Автомат по множеству встреченных букв

Для каждой вершины v исходного графа создадим 28 состояний:

- (v, S) — ещё не прошли ни одного ребра;
- (v, C_x) для каждой буквы x — пока встречалась только буква x ;
- (v, M) — уже встретились хотя бы две разные буквы.

Для каждого ребра $u \xrightarrow{c} v$ добавим переходы:

- $(u, S) \rightarrow (v, C_c)$;
- $(u, C_c) \rightarrow (v, C_c)$;
- $(u, C_x) \rightarrow (v, M)$ для всех $x \neq c$;
- $(u, M) \rightarrow (v, M)$.

Количество вершин автомата равно $28n$, количество рёбер — $28m$.

Стартовые состояния — все (v, S) , финишные — все (v, M) .

2. Находим L и проверяем бесконечность

Сделаем два обхода:

- из всех стартовых состояний;
- по обратным рёбрам из всех финишных состояний.

Оставим только вершины, которые достижимы из старта и из которых достигим финиш.

- Если в этом подграфе есть цикл, то можно сколько угодно раз обходить этот цикл и затем дойти до финиша. Значит $L = \infty$, а ответ равен -1 .
- Иначе это DAG.

На этом DAG'е посчитаем:

- $d_1[v]$ — максимальную длину пути из старта в вершину v ;
- $d_2[v]$ — максимальную длину пути из вершины v в финиш.

Тогда

$$L = \max d_1[v]$$

по всем финишным вершинам.

Если ни один финиш недостижим, то ответа вообще нет, и нужно вывести 0.

3. Оставляем только максимальные пути

Теперь нас интересуют только маршруты длины ровно L .

Оставим только те рёбра $u \rightarrow v$, для которых

$$d_1[u] + 1 + d_2[v] = L.$$

Именно эти рёбра лежат хотя бы на одном максимальном пути длины L .

После этого получаем новый DAG со свойством:

- все пути из старта в финиш имеют длину ровно L ;
- для каждого ребра $u \rightarrow v$ выполняется $d_1[v] = d_1[u] + 1$.

То есть граф становится **слоистым**.

4. Что нужно знать о префиксах и суффиксах

Для каждой вершины v этого DAG'а хотим знать:

- какие строки можно получить на путях из старта в v ;
- какие строки можно получить на путях из v в финиш, если читать их в обратном порядке.

Но хранить множество строк целиком не нужно. Достаточно трёх состояний:

- **NONE** — путей нет;
- **ONE(id)** — все такие строки одинаковы;
- **MANY** — существует хотя бы две разные строки.

Чтобы состояние **ONE(id)** было точным, а не вероятностным, построим бор всех возникающих строк.

- Пустой строке дадим идентификатор 0.
- Операция **extend(id, c)** возвращает идентификатор строки, полученной приписыванием буквы c справа.

Тогда две строки равны тогда и только тогда, когда равны их идентификаторы.

Слияние двух состояний определяется так:

- $\text{merge}(\text{NONE}, X) = X$;
- $\text{merge}(\text{MANY}, X) = \text{MANY}$;
- $\text{merge}(\text{ONE}(a), \text{ONE}(b)) = \text{ONE}(a)$, если $a = b$;
- $\text{merge}(\text{ONE}(a), \text{ONE}(b)) = \text{MANY}$, если $a \neq b$.

5. Считаем префиксы и суффиксы

Пусть $\text{pref}[v]$ — информация обо всех строках на путях из старта в v .

- Для стартовых вершин $\text{pref} = \text{ONE}(0)$.
- Дальше идём по слоям слева направо.
- Если есть ребро $u \xrightarrow{c} v$, то из состояния $\text{pref}[u]$ получаем кандидат для $\text{pref}[v]$ приписыванием буквы c .

Аналогично считаем $\text{suf}[v]$ — информацию обо всех **перевёрнутых** строках на путях из v в финиш:

- для финишных вершин $\text{suf} = \text{ONE}(0)$;
- идём по слоям справа налево;
- по ребру $u \xrightarrow{c} v$ переносим информацию из $\text{suf}[v]$ в $\text{suf}[u]$, тоже через $\text{extend}(\dots, c)$.

6. Проверяем середину

Теперь нужно понять, существует ли среди максимальных путей длины L непалиндром.

Случай 1: $L = 2k$. Тогда середина — это вершина слоя k .

Для такой вершины любой максимальный путь через неё имеет вид:

$$P + Q,$$

где $|P| = |Q| = k$.

Такой путь является палиндромом тогда и только тогда, когда строка P совпадает с перевёрнутой строкой Q .

Значит:

- если существует вершина v слоя k , для которой $\text{pref}[v]$ и $\text{suf}[v]$ **не являются одним и тем же** состоянием **ONE(id)**, то можно выбрать левую и правую половины так, чтобы получить непалиндром длины L ;
- иначе все максимальные пути являются палиндромами, и ответ равен $L - 1$.

Случай 2: $L = 2k + 1$. Тогда середина — это ребро $u \rightarrow v$, где $d_1[u] = k$.

Полностью аналогично:

- если существует такое среднее ребро, что $\text{pref}[u]$ и $\text{suf}[v]$ **не являются одним и тем же** состоянием $\text{ONE}(\text{id})$, то существует непалиндром длины L ;
- иначе все максимальные пути палиндромны, и ответ равен $L - 1$.

7. Почему проверка середины корректна

После фильтрации на максимальные пути граф стал слоистым, а все пути из старта в финиш имеют одинаковую длину L .

Значит палиндромность полностью определяется тем, совпадает ли левая половина пути с перевёрнутой правой половиной.

Если в некоторой середине с одной стороны возможны две разные строки, либо единственные возможные строки слева и справа различаются, то можно выбрать несовпадающие половины и получить непалиндром.

Если же в каждой середине возможна ровно одна строка слева и ровно одна строка справа, причём они совпадают, то любой максимальный путь будет палиндромом.

Время работы

Пусть $K = 28$ для полной задачи и $K = 4$ для бинарного алфавита. Тогда:

- число вершин автомата — $\mathcal{O}(Kn)$;
- число рёбер автомата — $\mathcal{O}(Km)$;
- все обходы, топологическая сортировка и динамики работают за $\mathcal{O}(K(n + m))$;
- бор строк тоже строится за линейное время от числа обработанных рёбер.

Итоговая сложность: $\mathcal{O}(K(n + m))$ по времени и памяти. Для полной задачи это $\mathcal{O}(28(n + m))$, что укладывается в ограничения.

Бонус

Эту задачу можно решить даже в случае, если ребра могут иметь произвольные веса, не ограничиваясь 26 буквами латинского алфавита, например натуральные числа от 1 до m . Для этого достаточно применить одну достаточно классическую идею: давайте построим коллекцию из не очень большого числа множеств $S_1, S_2, \dots, S_k$, что для каждой пары чисел $1 \leq x < y \leq n$, найдется k , что $x \in S_k, y \notin S_k$. Для этого достаточно взять как очередное множество все такие натуральные числа, что бит 2^k отсутствует в двоичном разбиении этого числа.

Таким образом мы можем $\log m$ раз запустить решение для двоичного алфавита.